

CONNECTIONS BETWEEN SCHEDULE-FREE OPTIMIZERS, AdEMAMix, AND ACCELERATED SGD VARIANTS

Anonymous authors
Anonymous submission

ABSTRACT

We contribute a unified accelerated-SGD interpretation of several recent deep-learning optimizers whose practical designs modify momentum. Although Schedule-Free optimizers, AdEMAMix, MARS, and Lion were introduced with different motivations, we show that their shared structure is a decoupling between the momentum coefficient and the weight assigned to the current gradient. Our first contribution is structural: Schedule-Free SGD is accelerated SGD followed by weight averaging, while Lion, MARS, Schedule-Free AdamW, and AdEMAMix can be read as accelerated-SGD-style updates combined with sign or preconditioning operations. Our second contribution is empirical: on a 150m-parameter language-modeling task, AdEMAMix and Simplified-AdEMAMix perform best in the small-batch regime, matching their closest structural alignment with accelerated SGD variants. Our third contribution clarifies the regime boundary: at large batch size, the acceleration advantage diminishes, and Schedule-Free AdamW underperforms AdamW with cosine decay because its momentum and weight-averaging coefficients are coupled. Finally, we propose Simplified-AdEMAMix, a single-momentum variant that matches AdEMAMix across the evaluated small- and large-batch settings while removing the need for two distinct momentum terms. The code for Simplified-AdEMAMix will be released upon acceptance.

1 INTRODUCTION

Recent optimizer proposals for deep learning—including Lion (Chen et al., 2023), ScheduleFreeSGD/AdamW (Defazio et al., 2024), and AdEMAMix (Pagliardini et al., 2024)—can look like independent algorithmic inventions. We organize them around a simpler reviewer-facing question: do their momentum modifications instantiate, approximate, or extend accelerated SGD mechanisms? The paper’s central claim is that the most salient commonality across these optimizers is not their implementation surface, but the way each separates the role of momentum from the role of the current gradient.

The theoretical reference point comes from acceleration under stochastic noise. Classical heavy-ball and Nesterov momentum accelerate deterministic gradient descent, especially for quadratic objectives, but they do not by themselves accelerate SGD (Jain et al., 2018; Liu & Belkin, 2020). Accelerated SGD methods for noisy settings instead use momentum schemes in which the momentum coefficient is decoupled from the coefficient multiplying the current gradient (Jain et al., 2018; Vaswani et al., 2019; Liu & Belkin, 2020; Gupta et al., 2023). That decoupling is the structural feature we surface throughout the paper.

The first contribution is an explicit mapping from modern optimizer updates to this accelerated-SGD template. We prove that Schedule-Free SGD is accelerated SGD followed by weight averaging, and we show how Lion, Schedule-Free AdamW, MARS, and AdEMAMix fit the same template after accounting for sign operations or preconditioning. Prior work has noted related momentum-and-averaging connections (Defazio, 2021), but the specific reviewer-useful map from these recent optimizers to accelerated SGD has not, to our knowledge, been stated in this form.

The second contribution tests whether the structure is predictive in language-model training. We train a 150m decoder-only transformer for 15b tokens with a small batch size of 32k tokens, a setting intended to emphasize the noise-dominated regime where accelerated-SGD effects should be most visible. There, Schedule-Free AdamW closely matches accelerated SGD-based AdamW (Algorithm 3), accelerated methods provide small gains, and AdEMAMix performs best among the compared optimizers. We then make the boundary of the contribution explicit by moving to large batch sizes, where the acceleration-related advantage diminishes.

The third contribution is an intentionally conservative simplification. Guided by the accelerated-SGD interpretation of AdEMAMix, we introduce Simplified-AdEMAMix, which keeps one momentum term and preserves AdEMAMix-level performance in both evaluated batch-size regimes. In summary, the paper contributes:

1. **Connection:** We establish precise theoretical links between accelerated SGD and recent optimizers, including Schedule-Free SGD, Lion, MARS, and AdEMAMix.
2. **Empirical validation:** We test AdamW, Schedule-Free AdamW, AdEMAMix, MARS, and related variants on a 150-million-parameter decoder-only transformer, and find that AdEMAMix, the method most closely aligned with accelerated SGD variants, performs best among the compared methods.
3. **Regime distinction:** We show that the gains associated with these accelerated-SGD-like methods diminish at large batch sizes relative to Adam, and that Adam with momentum scheduling can match AdEMAMix in this regime.
4. **Schedule-Free limitation:** We identify a high-batch-size failure mode of Schedule-Free AdamW relative to AdamW with cosine decay, and attribute it to the intrinsic coupling between momentum and weight averaging coefficients.
5. **Simplified optimizer:** We propose Simplified-AdEMAMix, which preserves AdEMAMix performance across the evaluated small- and large-batch regimes while removing the need for two distinct momentum terms.

2 RELATED WORK

We organize related work around the two lines that the paper connects: accelerated SGD theory, which explains why momentum decoupling can matter, and practical optimizer design, where similar decouplings have recently reappeared.

On the theory side, Jain et al. (2018) introduced an accelerated SGD variant with improved convergence rates for least-squares problems. Kidambi et al. (2018) simplified the update and showed formally that ordinary momentum does not provide acceleration in that setting. Later work (Liu & Belkin, 2020; Vaswani et al., 2019; Gupta et al., 2023) extended accelerated-SGD guarantees to broader convex and strongly convex settings under different assumptions. These results supply the template used here: acceleration is associated with a momentum update whose coefficient is not tied to the current-gradient coefficient in the final step.

On the optimizer-design side, several methods contain the same motif but were not always presented as accelerated-SGD relatives. Lucas et al. (2019) used weighted sums of multiple momentum terms with different coefficients, and Ma & Yarats (2019) proposed an optimizer explicitly motivated by Jain et al. (2018). Recent optimizers make the connection more salient: Lion (Chen et al., 2023) uses different weights for the gradient and momentum term before a sign operation, while AdEMAMix (Pagliardini et al., 2024) blends two momentum scales in the final update. Our contribution is to place these methods, along with Schedule-Free optimizers and MARS, into one accelerated-SGD-oriented comparison.

3 BACKGROUND

This section introduces the three primitives used to make the paper’s contributions legible: standard momentum, weight averaging, and the accelerated-SGD template. The later optimizer rewrites reuse this notation so that each contribution can be checked against the same baseline structure.

3.1 MOMENTUM

We begin with standard momentum to define the baseline that the later optimizer connections modify. For weights w_t and momentum coefficient β , the update is:

$$m_t = \beta m_{t-1} + \nabla f(w_t); \quad w_t = w_{t-1} - \eta m_t$$

3.2 WEIGHT AVERAGING

Weight averaging is the second ingredient needed for the Schedule-Free decomposition. In stochastic optimization it reduces iterate noise by returning a weighted average $\bar{w}_T$ rather than only the final iterate w_T , with weights denoted by γ_t :

$$\bar{w}_T = (1 - \gamma_T)\bar{w}_{T-1} + \gamma_T w_T$$

The averaging schemes relevant to our connection use coefficients γ_t of the form:

$$\gamma_t \approx 1 - \frac{1}{\delta t}$$

for some constant $0 \leq \delta \leq 1$.

3.3 ACCELERATED SGD

The main analytical frame is the following generalized accelerated-SGD form. This equation is the reference template used to make the later optimizer contributions easy to scan:

$$m_t = \beta_{a,t}m_{t-1} + g_t, \quad w_{t+1} = w_t - \eta_{a,t}m_t - \alpha_{a,t}g_t \quad (1)$$

Here, $\beta_{a,t}, \alpha_{a,t}, \eta_{a,t}$ are possibly time-dependent scalar coefficients, and g_t is the stochastic gradient evaluated at w_t . The subscript ‘a’ marks the accelerated-SGD parameterization used for the paper’s comparisons.

This form makes the central structural claim inspectable. Setting $\alpha_{a,t} = 0$ recovers standard SGD with momentum, while choosing nonzero $\alpha_{a,t}$ decouples the current-gradient contribution from the momentum coefficient. Prior accelerated SGD methods, including those of Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023), fit this template; Appendix B records the explicit equivalences.

4 CONNECTIONS BETWEEN EXISTING OPTIMIZERS AND ACCELERATED SGD

This section contains the paper’s main structural contribution: each optimizer is rewritten so that its relationship to the accelerated-SGD form is visible. The goal is not to claim that all methods are identical, but to isolate which part of each update corresponds to accelerated SGD and which part comes from weight averaging, preconditioning, or a sign operation. The section ends by using this structure to motivate Simplified-AdEMAMix, a single-momentum variant of AdEMAMix.

4.1 SCHEDULE-FREE SGD

Schedule-Free SGD (Defazio et al., 2024) is the cleanest case for our connection because its constant-learning-rate update separates into two recognizable pieces: an accelerated-SGD trajectory and an averaging trajectory. Using the notation of Defazio et al. (2024), the update is:

$$\begin{aligned} y_t &= (1 - \beta)z_t + \beta x_t \\ z_{t+1} &= z_t - \gamma g(y_t) \\ x_{t+1} &= (1 - c_{t+1})x_t + c_{t+1}z_{t+1} \end{aligned}$$

Here, y_t is the current model used to evaluate the gradient, while x_t is the model used for evaluation. To expose the accelerated-SGD component, we rewrite the update in terms of y_t and a momentum variable m_t , defined by:

$$m_{t+1} = \frac{x_t - z_{t+1}}{\gamma}. \quad (2)$$

The first contribution of the derivation is to show that this variable obeys a standard momentum recursion:

$$m_t = \frac{x_t - z_{t+1}}{\gamma} \quad (3)$$

$$= \frac{x_t + \gamma g_t - z_t}{\gamma} \quad (4)$$

$$= \frac{x_t - z_t + \gamma g_t}{\gamma} \quad (5)$$

$$= \frac{(1 - c_t)(x_{t-1} - z_t) + \gamma g_t}{\gamma} \quad (6)$$

$$= (1 - c_t)m_{t-1} + g_t. \quad (7)$$

Thus, m_t is exactly the momentum variable in Equation (1) with $\beta_{a,t} = 1 - c_t$. The corresponding update for y_t then reveals the current-gradient coefficient:

$$y_{t+1} = (1 - \beta)z_{t+1} + \beta x_{t+1} \quad (8)$$

$$= (1 - \beta)(z_t - \gamma g_t) + \beta((1 - c_{t+1})x_t + c_{t+1}z_{t+1}) \quad (9)$$

$$= (1 - \beta)z_t + \beta x_t - (1 - \beta)\gamma g_t + \beta c_{t+1}(z_{t+1} - x_t) \quad (10)$$

$$= y_t - \gamma[\beta c_{t+1}m_t + (1 - \beta)g_t]. \quad (11)$$

Therefore the y_t trajectory follows Equation (1) with $\eta_{a,t} = \gamma\beta c_{t+1}$ and $\alpha_{a,t} = \gamma(1 - \beta)$, taking $w_t = y_t$. This is the accelerated-SGD part of Schedule-Free SGD. The remaining question is what role the evaluation iterate x_t plays, so we rewrite its dynamics:

$$\begin{aligned} x_{t+1} &= (1 - c_{t+1})x_t + c_{t+1}z_{t+1} \\ x_{t+1} &= (1 - c_{t+1})x_t + c_2 \left(\frac{y_{t+1} - \beta x_{t+1}}{1 - \beta} \right) \\ x_{t+1}(1 - \beta + c_{t+1}\beta) &= (1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1} \\ x_{t+1} &= \frac{(1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1}}{(1 - c_{t+1})(1 - \beta) + c_{t+1}}. \end{aligned}$$

Thus, x_t is a weighted average of y_t . Recursively expanding x_t shows that it becomes an exponential average of y_t when c_t is constant. This proves the main Schedule-Free contribution of our analysis: Schedule-Free SGD is accelerated SGD followed by weight averaging.

This decomposition makes the benefits of Schedule-Free SGD easier to assign to two mechanisms:

1. Improved performance compared to standard SGD with momentum, due to its equivalence to accelerated SGD.
2. The ability to use a constant learning rate without scheduling, enabled by weight averaging (specifically, tailed weight averaging; see Section 4.1.3).

The decomposition also clarifies two implementation advantages that remain specific to Schedule-Free SGD/Adam:

- It does not require additional memory for weight averaging.
- It eliminates the need for an explicit weight averaging coefficient as a hyperparameter.

The same decomposition also identifies a limitation: Section 5.1 shows that coupling the momentum and weight-averaging coefficients does not scale well to large batch sizes.

4.1.1 CASE: $\beta = 0.0$

This edge case isolates the averaging contribution. As noted in (Defazio et al., 2024), $\beta = 0$ reduces Schedule-Free SGD to standard SGD with weight averaging, and because $c_t = 1/t$, averaging begins immediately.

4.1.2 CASE: $\beta = 1.0$

This edge case isolates the momentum contribution. As noted in (Defazio et al., 2024), $\beta = 1$ reduces Schedule-Free SGD to standard momentum SGD, with $\beta_{a,t}$ scaling as $1 - 1/t$.

4.1.3 CASE: $\beta = 0.9$

At the default Schedule-Free SGD setting $\beta = 0.9$, the connection implies the following concrete parameter behavior:

- As c_t scales as $1/t$, momentum grows as $1 - 1/t$.
- The ratio of the weight assigned to the current gradient versus momentum is fixed at $(1 - \beta)/(\beta c_{t+1}) \approx 0.11$.
- Weight averaging is applied approximately over the most recent 10% of the iterates.

4.2 LION

The update rule for Lion (Chen et al., 2023) is given by:

$$m'_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\theta_t = \theta_{t-1} - \eta \text{sign}(m'_t)$$

$$m_t = \beta_2 m_{t-1} + (1 - \beta_2) g_t.$$

Lion (Chen et al., 2023) therefore contributes a compact example of the same pattern: before the coordinate-wise sign operation, the update uses accelerated-SGD-style decoupling between the gradient term and the momentum coefficient.

4.3 MARS

MARS-Approx, the practical version of MARS (Yuan et al., 2024), contributes another intermediate case: it fits the accelerated-SGD pattern after a change of variables, but it also uses its own preconditioning rule. Ignoring bias correction and clipping, the update is:

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ m_t &= \beta_1 m_{t-1} + (1 - \beta_1) c_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \\ x_{t+1} &= x_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

Here, m_t and v_t are the first- and second-order momentum terms, and x_t denotes the model parameters. The accelerated-SGD structure becomes visible after defining $\hat{m}_t = m_t - \gamma g_t$:

$$\begin{aligned}
c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\
\hat{m}_t &= \beta_1 \hat{m}_{t-1} + (1 - \beta_1)(1 - \gamma)g_t \\
v_t &= \beta_2 v_{t-1} + (1 - \beta_2)c_t^2 \\
x_{t+1} &= x_t - \eta \frac{\hat{m}_t + \gamma g_t}{\sqrt{v_t} + \epsilon}
\end{aligned}$$

This rewrite shows that the momentum component follows the general accelerated-SGD framework, while the preconditioner remains MARS-specific rather than AdamW-like. The experiments in Section 5 therefore test both the shared accelerated-SGD structure and this distinct preconditioning choice.

4.4 ADEMAMix

AdEMAMix (Pagliardini et al., 2024) is the most contribution-salient optimizer in our comparison because it most closely resembles accelerated SGD-based AdamW. The difference is implementation form: accelerated SGD combines the current gradient and a momentum term directly, whereas AdEMAMix keeps two momentum terms with different coefficients and combines them linearly. Algorithm 1 states the update.

To make the correspondence explicit, we analyze the $\beta_1 = 0$ variant of AdEMAMix. Pagliardini et al. (2024) showed that this variant performs nearly as well as the full optimizer at small batch sizes, and our experiments in Section 5 support the same conclusion. With $\beta_1 = 0$, AdEMAMix fits the general accelerated-SGD framework in Equation (1). Moreover, its schedules for β_3 , the momentum coefficient, and α , the relative weight on the current-gradient component, closely track theoretical schedules proposed for accelerated SGD (Gupta et al., 2023).

The relevant theoretical schedule is visible in smooth convex optimization, where stochastic acceleration requires a momentum scheme of the form:

$$\beta_{a,t} = 1 - \frac{k}{t}$$

for some constant $k > 0$, as established by Gupta et al. (2023). AdEMAMix approximately follows this contribution-relevant schedule by increasing β_3 over training.

The comparison also requires tracking how the momentum buffer is normalized. In accelerated SGD, momentum is maintained as:

$$m_t = \beta_{a,t} m_{t-1} + g_t$$

whereas Algorithm 1 uses the EMA-style update:

$$m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}.$$

When $\beta_{a,t}$ scales as $1 - 1/t$, the accumulated contribution of past gradients in accelerated-SGD momentum grows proportionally to t . AdEMAMix mirrors this effect because its coefficient α also scales proportionally to t . This structural match motivates the empirical result in Figure 1, where AdEMAMix outperforms the other compared optimizers.

The same analysis points to a large-batch limitation. As reported in Pagliardini et al. (2024), AdEMAMix degrades at large batch sizes when $\beta_1 = 0.0$. The theory in Gupta et al. (2023) suggests that the current-gradient weight should decrease as batch size increases, but AdEMAMix keeps a fixed weight of 1 on the current gradient. This mismatch explains why the small-batch interpretation does not transfer unchanged to the large-batch regime.

This diagnosis leads directly to Simplified-AdEMAMix. The variant keeps an explicit weight on the current gradient, removes the need to schedule α , and maintains only one momentum term. The experiments then test the intended contribution: matching AdEMAMix across both small- and large-batch setups with a simpler optimizer.

Algorithm 1 Single step of AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters β_3, α . Warmup parameter T_{α, β_3} , note that we usually set it to T . β_{start} is usually set to β_1 .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_3^{(t)} \leftarrow f_{\beta_3}(t, \beta_3, \beta_{\text{start}}, T_{\alpha, \beta_3})$ and $\alpha^{(t)} \leftarrow f_{\alpha}(t, \alpha, T_{\alpha, \beta_3})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the fast EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + (1 - \beta_1) g^{(t)}$
 - 6: Update the slow EMA m_2 : $m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}$
 - 7: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 8: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{\hat{m}_1^{(t)} + \alpha^{(t)} m_2^{(t)}}{\sqrt{\hat{\nu}^{(t)} + \epsilon}} \right)$
-

Algorithm 2 Single step of Simplified AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters $\alpha, \beta_{\text{start}}$. Warmup parameter T_{β_1} , note that we usually set it to T .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_1^{(t)} \leftarrow f_{\beta_1}(t, \beta_1, \beta_{\text{start}}, T_{\beta_1})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + g^{(t)}$
 - 6: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 7: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{m_1^{(t)} + \alpha g^{(t)}}{\sqrt{\hat{\nu}^{(t)} + \epsilon}} \right)$
-

Algorithm 3 Single step of accelerated SGD based Adam with weight averaging. For simplicity we ignore the initialization, other boundary effects such as bias correction, and weight decay. Hyperparameters: Learning rate η , betas = $(\beta_1, \beta_2, \beta_3)$, weight averaging coefficient δ , and epsilon ϵ .

- 1: Sample batch B_t .
- 2: $g \leftarrow -\nabla_w \phi_{B_t}(w_t)$
- 3: $v \leftarrow \beta_2 v + (1 - \beta_2)(g \odot g)$
- 4: $N \leftarrow \frac{\beta_3 m + (1 - \beta_3)g}{\sqrt{v} + \epsilon}$
- 5: $w \leftarrow w - \eta N$
- 6: $m \leftarrow \beta_1 m + (1 - \beta_1)g$
- 7: $c = \max(1 - 1/t, 1 - 1/(\delta t))$
- 8: $w_{\text{avg}} \leftarrow c w_{\text{avg}} + (1 - c)w$

4.5 SIMPLIFIED ADEMAMIX

Simplified-AdEMAMix is the algorithmic outcome of the preceding connections. It removes one of AdEMAMix’s two momentum terms and avoids scheduling α , while preserving the accelerated-SGD-style separation between the momentum component and the current-gradient component. Algorithm 2 gives the update using theory-style momentum rather than EMA-style momentum. The final update assigns a fixed weight α to the gradient, and $\alpha = 0$ recovers standard Adam after the corresponding transformations of η and β_1 . Section 5 evaluates whether this simpler structure preserves AdEMAMix performance in both tested batch-size regimes.

5 EXPERIMENTS

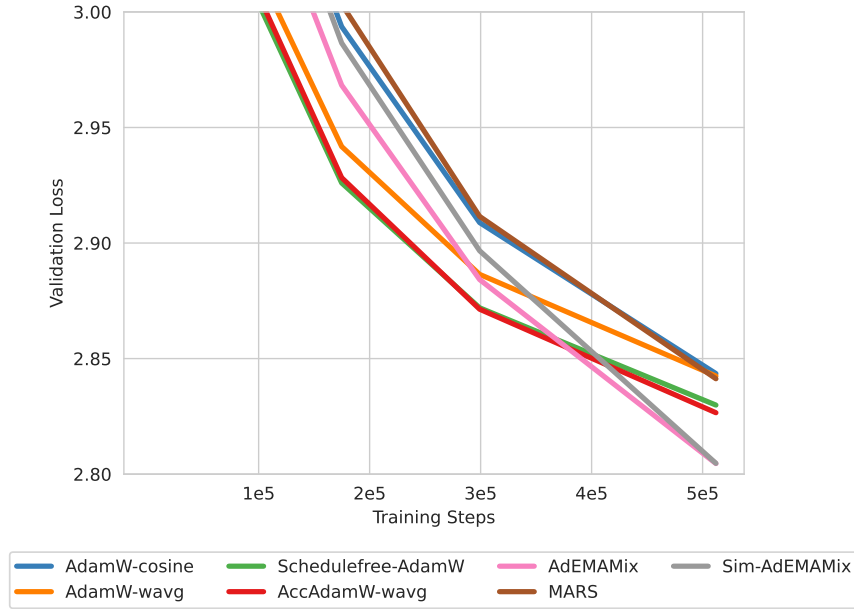


Figure 1: Small-batch language-modeling comparison on a 150m decoder-only transformer. The best runs highlight the empirical contribution: AdEMAMix and Simplified-AdEMAMix perform best among the tested optimizers, consistent with their close structural similarity to accelerated SGD variants discussed in Section 5.

The experiments are designed to validate the paper’s contribution structure, not to introduce a new benchmark. We train a 150-million-parameter decoder-only transformer on C4 for language modeling with sequence length 1024, batch size 32, and 15 billion tokens ($\approx 5 \times$ Chinchilla). This small-batch setting keeps optimization in a noise-dominated regime, where the accelerated-SGD interpretation predicts that differences among the optimizers should be most visible.

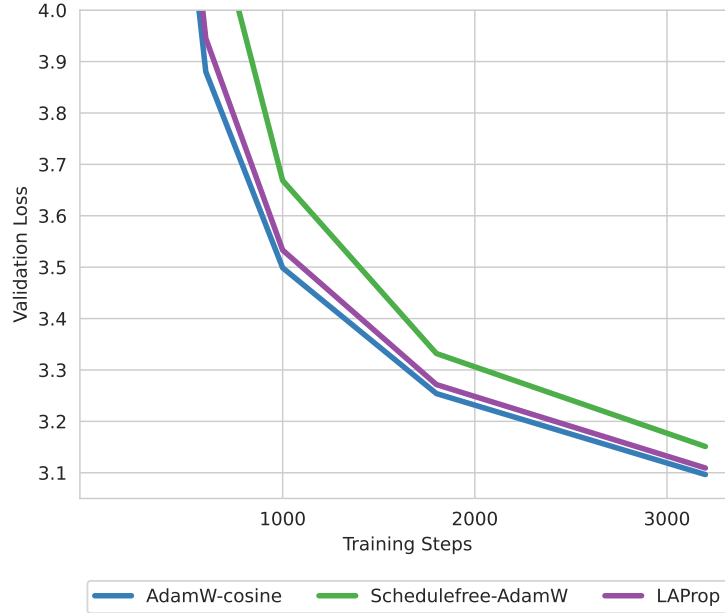


Figure 2: Large-batch comparison of AdamW with cosine decay, Schedule-Free AdamW, and LARProp. The result highlights the regime-boundary contribution by isolating the high-batch-size limitation of Schedule-Free AdamW discussed in Section 5.1.

We compare optimizers chosen to instantiate the theoretical categories introduced above:

1. Standard AdamW with cosine decay
2. Standard AdamW with weight averaging
3. Schedule-Free AdamW
4. Accelerated AdamW with weight averaging (Algorithm 3)
5. MARS
6. AdEMAMix
7. Simplified-AdEMAMix

Section A reports the hyperparameter sweeps, making the result comparison reviewer-checkable under the same tuning budget.

Figure 1 supports the first empirical contribution. Schedule-Free AdamW and Accelerated AdamW with tailed weight averaging perform similarly, which is the behavior predicted by the Schedule-Free decomposition. Both methods outperform AdamW with cosine decay and AdamW with tailed weight averaging in this setting. The most salient result is that AdEMAMix and Simplified-AdEMAMix outperform all compared methods, consistent with their closer alignment to accelerated SGD variants.

5.1 LARGE BATCH SIZE EXPERIMENTS

The large-batch experiments make the boundary of the accelerated-SGD interpretation explicit. After using small batches to isolate the noise-dominated regime, we train with a batch size of 1 million tokens for 3 billion tokens ($\approx$ Chinchilla scale) and ask which contribution claims persist when gradient noise is reduced.

Schedule-Free AdamW: Figure 2 shows the limitation predicted by our decomposition: Schedule-Free AdamW performs substantially worse than AdamW at large batch size. We attribute the gap to the coupling between weight

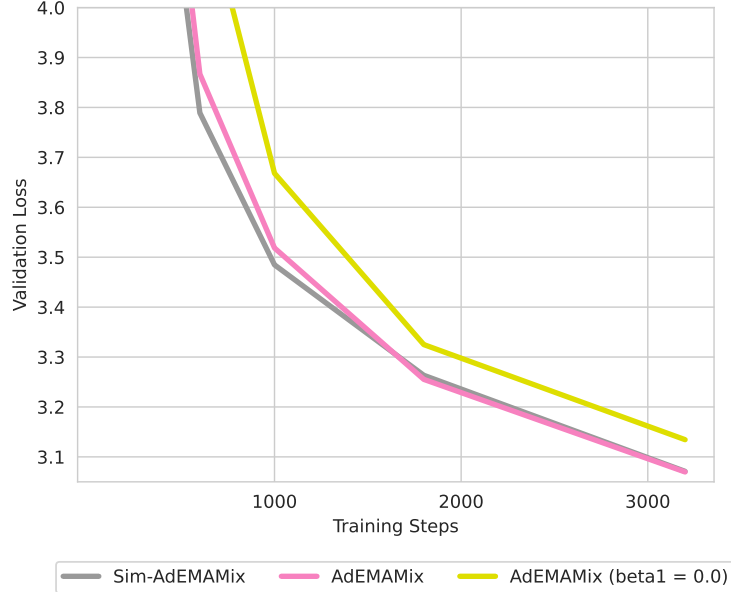


Figure 3: Large-batch comparison of AdEMAMix with and without $\beta_1 = 0.0$ and Simplified-AdEMAMix. The figure supports the simplification contribution by testing whether the single-momentum variant preserves AdEMAMix performance in Section 5.1.

averaging and momentum coefficients. In this regime, the preferred momentum value is far below $1 - 1/t$. A coefficient of the form $\approx 1 - r/t$ with $r \geq 1$ can reduce momentum, but increasing r also shortens the effective weight-averaging window, so the two design choices cannot be tuned independently.

We also rule out a competing explanation for this result. AdamW and Schedule-Free AdamW apply momentum and preconditioning in different orders: AdamW applies momentum before preconditioning, whereas Schedule-Free AdamW applies preconditioning before momentum, making it algorithmically similar to LProp (Ziyin et al., 2021). Figure 2 shows that AdamW and LProp perform comparably, so this ordering difference is unlikely to explain the Schedule-Free AdamW gap.

AdEMAMix: Figure 3 reproduces the large-batch behavior reported by Pagliardini et al. (2024): setting $\beta_1 = 0.0$ causes AdEMAMix to degrade relative to the two-momentum version. The accelerated-SGD framing makes this failure mode interpretable. AdEMAMix keeps a fixed weight of 1 on the current gradient, while accelerated SGD theory (Gupta et al., 2023) suggests that this weight should diminish as batch size increases.

Figure 3 also supports the simplification contribution. Simplified-AdEMAMix matches AdEMAMix while avoiding two separate momentum terms. In the best large-batch setting, this match occurs at $\alpha = 0.0$, so Simplified-AdEMAMix reduces to standard Adam equipped with momentum scheduling.

6 CONCLUSION

This paper contributes a unified reading of several recent optimizers through accelerated SGD. We show that Schedule-Free SGD is accelerated SGD followed by weight averaging, and that Lion, MARS, Schedule-Free AdamW, and AdEMAMix can be understood as accelerated-SGD-style updates combined with sign operations or preconditioning. This framing makes the role of momentum decoupling explicit and clarifies why AdEMAMix is the closest practical match to the accelerated-SGD variants considered here.

The empirical contribution supports both the usefulness and the limits of this framing. In small-batch language-model training, AdEMAMix and Simplified-AdEMAMix perform best among the tested methods, while Schedule-Free AdamW tracks accelerated AdamW with weight averaging. In large-batch training, the acceleration-related

advantage diminishes, and Schedule-Free AdamW suffers from its coupling of momentum and weight averaging. The algorithmic contribution is Simplified-AdEMAMix: it retains the AdEMAMix connection with a single-momentum optimizer that matches AdEMAMix across the tested regimes while avoiding the extra momentum buffer.

REFERENCES

- Chen, X., Liang, C., Huang, D., Real, E., Wang, K., Pham, H., Dong, X., Luong, T., Hsieh, C., Lu, Y., and Le, Q. V. Symbolic discovery of optimization algorithms. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*. URL http://papers.nips.cc/paper_files/paper/2023/hash/9a39b4925e35cf447ccba8757137d84f-Abstract-Conference.html.
- Defazio, A. Momentum via primal averaging: Theoretical insights and learning rate schedules for non-convex optimization, 2021. URL <https://arxiv.org/abs/2010.00406>.
- Defazio, A., Yang, X., Mehta, H., Mishchenko, K., Khaled, A., and Cutkosky, A. The road less scheduled. *CoRR*, abs/2405.15682, 2024. doi: 10.48550/ARXIV.2405.15682. URL <https://doi.org/10.48550/arXiv.2405.15682>.
- Gupta, K., Siegel, J., and Wojtowysch, S. Achieving acceleration despite very noisy gradients, 2023. URL <https://arxiv.org/abs/2302.05515>.
- Jain, P., Kakade, S. M., Kidambi, R., Netrapalli, P., and Sidford, A. Accelerating stochastic gradient descent for least squares regression. In Bubeck, S., Perchet, V., and Rigollet, P. (eds.), *Proceedings of the 31st Conference On Learning Theory*, volume 75 of *Proceedings of Machine Learning Research*, pp. 545–604. PMLR, 06–09 Jul 2018. URL <https://proceedings.mlr.press/v75/jain18a.html>.
- Kidambi, R., Netrapalli, P., Jain, P., and Kakade, S. M. On the insufficiency of existing momentum schemes for stochastic optimization. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=rJTutzbA->.
- Liu, C. and Belkin, M. Accelerating sgd with momentum for over-parameterized learning. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=rlgixp4FPH>.
- Lucas, J., Sun, S., Zemel, R., and Grosse, R. Aggregated momentum: Stability through passive damping. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Syxt5oC5YQ>.
- Ma, J. and Yarats, D. Quasi-hyperbolic momentum and adam for deep learning. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=SlfUpoR5FQ>.
- Pagliardini, M., Ablin, P., and Grangier, D. The ademamix optimizer: Better, faster, older. 2024. URL <https://arxiv.org/abs/2409.03137>.
- Porian, T., Wortsman, M., Jitsev, J., Schmidt, L., and Carmon, Y. Resolving discrepancies in compute-optimal scaling of language models. *CoRR*, abs/2406.19146, 2024. doi: 10.48550/ARXIV.2406.19146. URL <https://doi.org/10.48550/arXiv.2406.19146>.
- Vaswani, S., Bach, F., and Schmidt, M. Fast and faster convergence of sgd for over-parameterized models and an accelerated perceptron. In Chaudhuri, K. and Sugiyama, M. (eds.), *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pp. 1195–1204. PMLR, 16–18 Apr 2019. URL <https://proceedings.mlr.press/v89/vaswani19a.html>.
- Yuan, H., Liu, Y., Wu, S., Zhou, X., and Gu, Q. Mars: Unleashing the power of variance reduction for training large models, 2024. URL <https://arxiv.org/abs/2411.10438>.
- Ziyin, L., Wang, Z. T., and Ueda, M. Laprop: Separating momentum and adaptivity in adam, 2021. URL <https://arxiv.org/abs/2002.04839>.

A HYPERPARAMETERS

For reproducibility of the small-batch contribution claims, we report the complete tuning ranges used for each optimizer:

1. AdamW with cosine decay and 51.2k warmup: learning rate in $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3]$, β_1 in $[0.9, 0.95]$, and β_2 in $[0.99, 0.999, 0.99968, 0.9999]$. The best values were $\beta_1 = .9$ and $\beta_2 = .999$, matching the defaults. Although larger batch sizes often use $\beta_2 = .95$, the benefit of higher β_2 at smaller batch sizes has also been observed by Porian et al. (2024).
2. AdamW with cosine decay and 10k warmup: learning rate in $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3]$, with $\beta_1 = 0.9$ and $\beta_2 = 0.999$ fixed to the best values from the previous sweep. This setting performed worse than the 51.2k-step warmup.
3. AdamW with constant-fraction weight averaging: learning rate in $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3]$, $\beta_1 = 0.9$, β_2 in $[0.99, 0.997, 0.999, 0.9997]$, δ in $[0.05, 0.1, 0.2]$.
4. AdamW with cosine decay and weight averaging: learning rate in $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and δ in $[0.025, 0.05, 0.1]$.
5. Accelerated SGD based AdamW with cosine decay: learning rate in $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3]$, β_1 in $[0.999, 0.99968, 0.9999]$, β_2 in $[0.99, 0.9968, 0.999]$, and $\beta_3 = 0.9$.
6. Accelerated SGD based AdamW with constant learning rate and weight averaging: learning rate in $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3]$, β_1 in $[0.99684, 0.999]$, β_2 in $[0.999]$, $\beta_3 = 0.9$, and δ in $[0.05, 0.1]$.
7. Accelerated SGD based AdamW with cosine decay and weight average: learning rate in $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3]$, β_1 in $[0.99684, 0.999]$, $\beta_2 = 0.999$, δ in $[0.05, 0.1]$, and $\beta_3 = 0.9$.
8. Schedule-Free AdamW with constant learning rate: learning rate in $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3, 1\text{e-}2]$, β_1 in $[0.8, 0.9, 0.95]$, and $\beta_2 = 0.999$.
9. Schedule-Free AdamW with cosine decay: learning rate in $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3, 1\text{e-}2]$, β_1 in $[0.8, 0.9, 0.95]$, and $\beta_2 = 0.999$.
10. MARS: learning rate in $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3, 1\text{e-}2]$, β_1 in $[0.9, 0.95, 0.99]$, β_2 in $[0.99, 0.999]$, γ in $[0.0, 0.01, 0.02, 0.03, 0.04, 0.05]$, with precondition 1d set to True.
11. AdEMAMix: learning rate in $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.999$, β_3 in $[0.99, 0.999, 0.9999]$, and α in $[2, 4, 8, 16]$.
12. Sim-AdEMAMix: learning rate in $[1\text{e-}6, 3.16\text{e-}6, 1\text{e-}5, 3.16\text{e-}5]$, β_1 in $[0.99, 0.999, 0.9999]$, $\beta_2 = 0.999$, and α in $[10, 20, 50, 100]$.

For reproducibility of the large-batch regime comparison, we report the corresponding tuning ranges:

1. Schedule-Free AdamW: learning rate in $[1\text{e-}3, 3.16\text{e-}3, 1\text{e-}2]$, β in $[0.8, 0.9, 0.95]$, β_2 in $[0.9, 0.95]$, and r in $[0.0, 5.0, 9.0, 50.0]$.
2. AdamW: learning rate in $[1\text{e-}3, 3.16\text{e-}3, 1\text{e-}2]$, β_1 in $[0.9, 0.95]$, and β_2 in $[0.9, 0.95]$.
3. LProp: learning rate in $[1\text{e-}3, 3.16\text{e-}3, 1\text{e-}2]$, β_1 in $[0.9, 0.95]$, and β_2 in $[0.9, 0.95]$.
4. AdEMAMix: learning rate in $[1\text{e-}3, 3.16\text{e-}3, 1\text{e-}2]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.95$, β_3 in $[0.9, 0.95, 0.99]$, and α in $[2, 4, 8, 16]$.
5. Sim-AdEMAMix: learning rate in $[1\text{e-}4, 3.16\text{e-}4, 1\text{e-}3]$, β_1 in $[0.9, 0.95, 0.99]$, $\beta_2 = 0.95$, and α in $[0.0, 0.5, 1.0]$.

B EQUIVALENCE OF PREVIOUS ACCELERATION METHODS

This appendix supports the paper’s connection contribution by showing that the accelerated-SGD template in Equation (1) covers the prior acceleration methods used as reference points. We rewrite the methods of Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023) into the same form.

B.1 AGNES

For AGNES, the starting update from Gupta et al. (2023) is:

$$x'_n = x_n + \alpha v_n \quad x_{n+1} = x'_n - \eta g'_n \quad v_{n+1} = \rho_n(v_n - g'_n)$$

where g'_n is the stochastic gradient evaluated at x'_n and the final function is evaluated at x_n . The accelerated-SGD form appears after rewriting the equations as

$$x'_{n+1} = x'_n - \eta g'_n + \alpha v_{n+1} \quad -\frac{v_{n+1}}{\rho_n} = \rho_{n-1} \left(-\frac{v_n}{\rho_{n-1}} \right) + g'_n$$

Thus x'_{n+1} follows an update of the form in Equation (1).

B.2 ASGD

For ASGD, the update of Jain et al. (2018) is:

$$y_{j-1} = \alpha x_{j-1} + (1 - \alpha)v_{j-1} \quad x_j = y_{j-1} - \delta g_{j-1} \quad z_{j-1} = \beta y_{j-1} + (1 - \beta)v_{j-1} \quad v_j = z_{j-1} - \gamma g_{j-1}$$

where g_{j-1} is the stochastic gradient evaluated at y_{j-1} and the function is evaluated on the tail-averaged x trajectory. Rewriting gives:

$$y_j = y_{j-1} - \alpha \delta g_{j-1} - (1 - \alpha)[y_{j-1} - v_j] \quad \frac{y_{j-1} - v_j}{\gamma - (1 - \beta)\alpha\delta} = (1 - \beta)\alpha \frac{y_{j-2} - v_{j-1}}{\gamma - (1 - \beta)\alpha\delta} + g_{j-1}$$

These equations therefore follow the form of Equation (1).

B.3 MASS

For MaSS, the update of Liu & Belkin (2020) is:

$$w_{t+1} = u_t - \eta_1 g_t \quad u_{t+1} = (1 + \gamma)w_{t+1} - \gamma w_t + \eta_2 g_t$$

where g_t is the stochastic gradient evaluated at u_t and the function is evaluated at w_t . The same template appears after rewriting:

$$u_{t+1} = u_t - \gamma(w_t - u_t) - [\eta_1(1 + \gamma) - \eta_2]g_t \quad \frac{w_t - u_t}{\eta_1\gamma - \eta_2} = \gamma \frac{w_{t-1} - u_{t-1}}{\eta_1\gamma - \eta_2} + g_t$$

These equations therefore follow the form of Equation (1).

B.4 SGD WITH NESTEROV ACCELERATION

For SGD with Nesterov acceleration, the update of Vaswani et al. (2019) is:

$$w_{k+1} = \zeta_k - \eta g_k \quad \zeta_k = \alpha_k v_k + (1 - \alpha_k)w_k \quad v_{k+1} = \beta_k v_k + (1 - \beta_k)\zeta_k - \gamma_k \eta g_k$$

These equations expose the accelerated-SGD structure after the following rewrite:

$$\zeta_{k+1} = \zeta_k - \eta g_k + \alpha_{k+1}[v_{k+1} - w_{k+1}]; \quad v_{k+1} - w_{k+1} = \beta_k(1 - \alpha_k)[v_k - w_k] - \eta(\gamma_k - 1)g_k$$